FRANQ

Prezado(a) Candidato(a),

Os desafios técnicos apresentados a seguir foram elaborados para avaliar suas habilidades como Engenheiro(a) de IA. Esperamos que você **escolha apenas um (1) dos dois desafios (Desafio Técnico 1 ou Desafio Técnico 2)** para implementar completamente, seguindo todos os requisitos e entregáveis especificados. Em relação ao desafio não escolhido, solicitamos que você apenas **pense no sistema arquitetural**. Os detalhes e aprofundamento sobre a arquitetura do segundo desafio serão o foco de discussão durante a entrevista técnica.

Desafio Técnico 1 – Engenheiro(a) de IA (Pleno)

Objetivo

Desenvolver um **Assistente Virtual de Dados** inteligente, capaz de atuar como um analista júnior para responder perguntas de negócio de forma independente.
O foco deste desafio é avaliar sua capacidade de criar sistemas que não apenas "seguem um roteiro", mas que conseguem **navegar por incertezas**, **buscar as próprias respostas** em um banco de dados e **apresentar os resultados visualmente**.

Cenário

A diretoria da empresa precisa de respostas rápidas sobre a operação. Atualmente, cada pergunta nova exige que um engenheiro escreva uma query SQL específica.
Precisamos de uma solução onde o diretor possa perguntar em linguagem natural e o sistema seja capaz de investigar o banco de dados fornecido (`anexo_desafio_1.db`) para formular uma resposta completa, incluindo gráficos ou tabelas quando pertinente.

Exemplos de perguntas esperadas:

- "Liste os 5 estados com maior número de clientes que compraram via app em maio."
- "Quantos clientes interagiram com campanhas de WhatsApp em 2024?"
- "Quais categorias de produto tiveram o maior número de compras em média por cliente?"
- "Qual o número de reclamações não resolvidas por canal?"
- "Qual a tendência de reclamações por canal no último ano?"

Recursos

Você receberá um arquivo SQLite (`clientes_completo.db`) contendo a seguinte estrutura já populada:

Tabela	Colunas
clientes	`id` (INT), `nome` (TEXT), `email` (TEXT), `idade` (INT), `cidade` (TEXT), `estado` (TEXT), `profissao` (TEXT), `genero` (TEXT).
compras	`id` (INT), `cliente_id` (INT), `data_compra` (TEXT), `valor` (REAL), `categoria` (TEXT), `canal` (TEXT).
suporte	`id` (INT), `cliente_id` (INT), `data_contato` (TEXT), `tipo_contato` (TEXT), `resolvido` (BOOLEAN), `canal` (TEXT).
campanhas_marketing	`id` (INT), `cliente_id` (INT), `nome_campanha` (TEXT), `data_envio` (TEXT), `canal` (TEXT), `interagiu` (BOOLEAN).

O que você deve fazer

1. O Motor de Inteligência

Crie uma aplicação backend (Python) que receba a pergunta do usuário e devolva a resposta consultando o banco de dados.

O sistema deve ser robusto o suficiente para lidar com:

- **Perguntas complexas**: Que podem exigir mais de uma consulta ou passos intermediários de raciocínio.
- **Erros de execução**: Se o sistema gerar SQL inválido, ele deve ser capaz de perceber o erro e o tentar corrigir sozinho.
- **Descoberta**: O sistema não deve depender de queries "hardcoded". Ele deve entender a estrutura do banco (schema) dinamicamente.

2. Interface (Frontend)

Construa uma interface simples (ex: Streamlit) onde o usuário possa interagir.

- **Visualização Dinâmica**: A interface deve ser capaz de mostrar os dados da forma mais apropriada para a pergunta (ex: exibir uma **Tabela** para listas, ou um **Gráfico de Linha/Barra** para tendências e comparações) ou de acordo com o que for pedido pelo usuário.
- **O "Raciocínio"**: A aplicação deve transparência sobre *como* chegou àquela conclusão (Mostrar os passos ou queries executadas).

Entregáveis

- Código-fonte completo e organizado em um repositório aberto no GitHub.
- `README.md` contendo:
    - Instruções de execução.

    - Explicação do fluxo de agentes e arquitetura escolhida.
    - Exemplos de consultas testadas.
    - Sugestões de melhorias ou extensões do projeto.

Stack sugerida

- **Python**
- **LangChain / LangGraph**
- **SQLite**
- **Streamlit**

Prazo

5 dias corridos a partir do envio deste desafio.

Desafio Técnico 2 – Engenheiro(a) de IA (Pleno)

Objetivo

Criar uma solução robusta para ingestão, classificação e extração de informações a partir de documentos não estruturados. Este desafio busca avaliar sua capacidade de arquitetar pipelines de dados integrados com LLMs, priorizando **confiabilidade**, **manutenibilidade** e **eficiência de custos**.

Cenário

A empresa possui um backlog de milhares de documentos digitalizados de diferentes tipos. Precisamos de uma solução automatizada para processar esses documentos e alimentar nosso ERP com dados estruturados.

O que você deve fazer

1. Arquivos de Entrada (fornecidos)

Os arquivos de teste já estão disponíveis no zip enviado. São **50 arquivos PDF** que simulam documentos digitalizados.

Os documentos estão diversificados entre 3 tipos:

- **Nota Fiscal:** Contém dados como fornecedor, CNPJ, data, lista de itens (descrição, qtd, valor) e valor total.
- **Contrato de Prestação de Serviços**: Contém partes (contratante/contratado), objeto do contrato, data de vigência e valor mensal.
- **Relatório de Manutenção**: Contém data, técnico responsável, equipamento, descrição do problema e solução aplicada.

2. Pipeline de Processamento

Crie uma aplicação/script que processe todos os arquivos de uma pasta com os arquivos `data/raw`. O sistema deve ser capaz de:

1. **Ingestão**: Ler os arquivos da pasta de entrada.
2. **Classificação**: Identificar automaticamente o tipo de cada documento (Nota Fiscal, Contrato ou Relatório).
3. **Roteamento**: Direcionar o documento para o fluxo de extração adequado.
4. **Extração Estruturada**: Extrair os campos específicos de cada tipo de documento (conforme definido na seção anterior) garantindo que a saída siga um formato JSON estrito.
5. **Persistência**: Salvar os dados extraídos de forma organizada (arquivo JSON consolidado, CSV ou banco de dados).

Requisitos Não-Funcionais

- **Eficiência**: Considere que este pipeline poderá escalar para milhões de documentos. A solução deve ser otimizada para latência e custo.
- **Robustez**: O pipeline não deve falhar completamente se encontrar um arquivo anômalo. Implemente tratamento de erros adequado.
- **Reprodutibilidade**: A execução deve produzir resultados consistentes.

Entregáveis

- Código fonte completo em um repositório aberto no GitHub.
- `README.md` contendo:
    - Instruções de configuração e execução.
    - **Justificativa da Arquitetura**: Explique por que você escolheu essa abordagem técnica em detrimento de outras

Stack sugerida

- **Python**
- **LangChain** / **LangGraph**

Prazo

5 dias corridos a partir do envio deste desafio.